

AGENTIC AI CHALLENGE · ROUND 2

Maintenance Wizard

Agentic predictive-maintenance decision support for heavy steel plants

A System of Action — it predicts what will break, prepares the fix, and refuses to close a job until every compliance step is logged.



CRITICAL · HIGH · MEDIUM · LOW priority-banded triage

Consolidated-Brain agent · runs on numpy + pandas + Streamlit alone

THE PROBLEM

Fragmented data, manual diagnosis, costly downtime

In a steel plant, an engineer diagnosing a fault juggles five disconnected systems — by hand, under time pressure, dependent on expert judgement. The result is slow, inconsistent decisions and avoidable unplanned downtime.

5 → 1

five fragmented sources collapsed into one traceable decision

1

Equipment manuals & SOPs

torque specs, isolation steps

2

Sensor / condition data

temperature, vibration, pressure...

3

Historical delay logs

downtime, tonnage lost

4

Failure / incident records

root cause, resolution

5

ERP spares inventory

stock levels & procurement lead time

Not a System of Record — a System of Action

SYSTEM OF RECORD

Passive databases

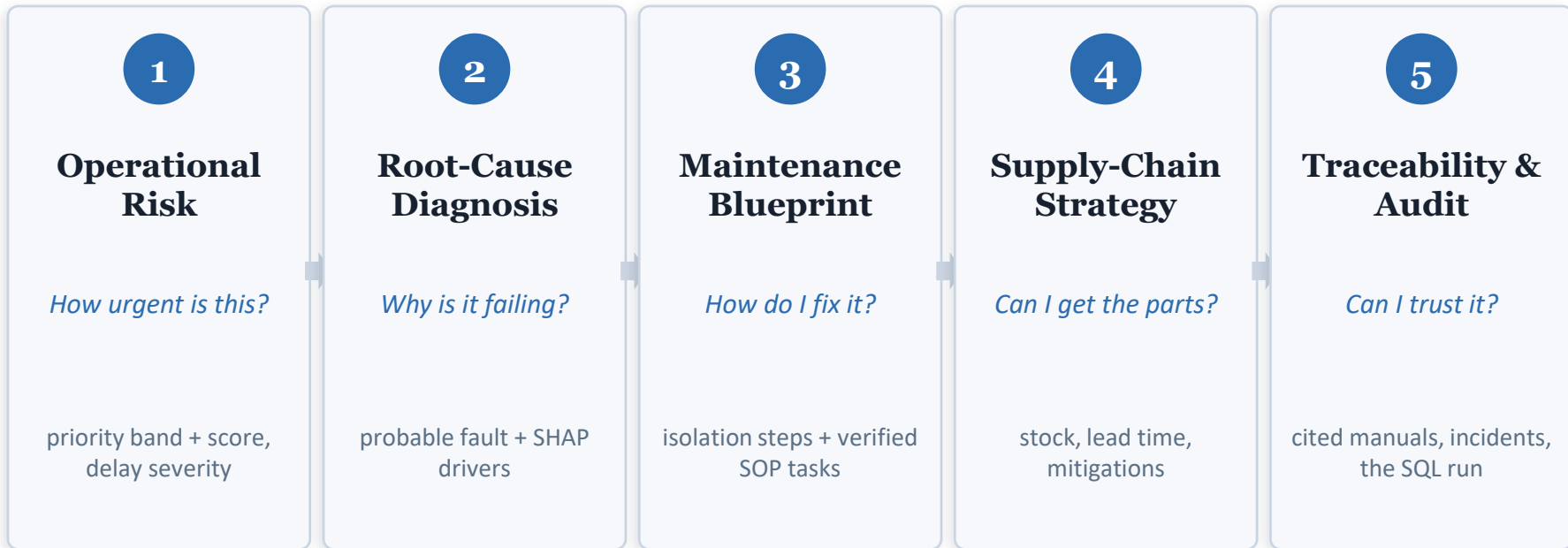
- Tells you what broke and when
- Waits for a human to ask
- Knowledge scattered across silos
- Reactive troubleshooting

SYSTEM OF ACTION

An autonomous agent

- Predicts what will break — and when
- Prepares the work order before login
- Fuses all five sources in one loop
- Closes the loop: blocks non-compliant jobs

One decision, five answered questions

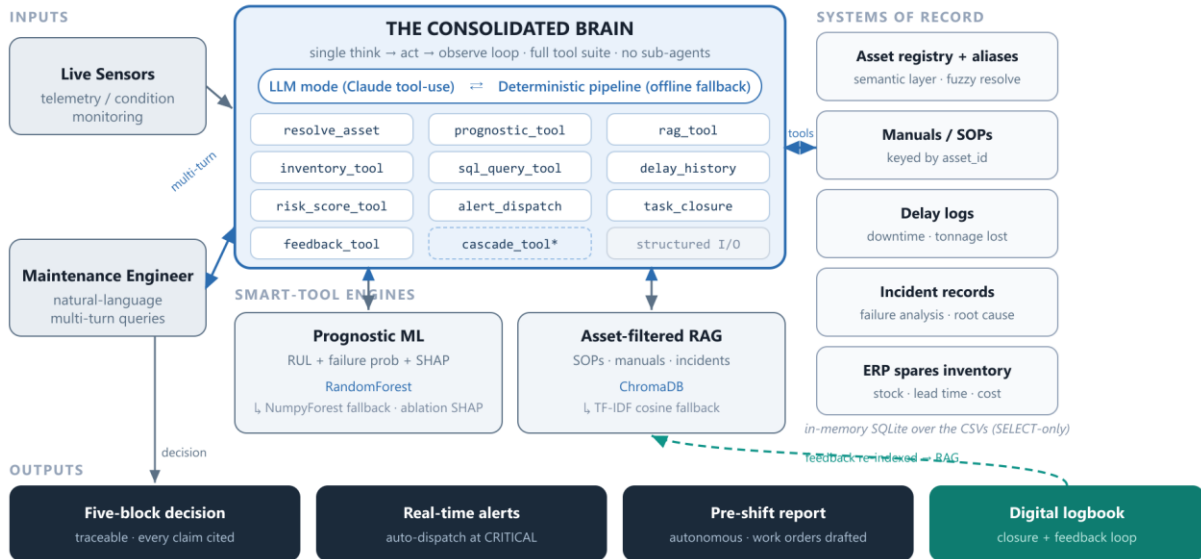


Every diagnosis returns the same mandatory five-block output — scannable, structured, and fully cited.

The Consolidated Brain

Maintenance Wizard — Consolidated-Brain Architecture

One traceable maintenance decision from five fused data sources · single agent loop · runs anywhere



Zero-dependency fallback ladder: runs on numpy + pandas + streamlit alone; auto-upgrades to scikit-learn + SHAP + ChromaDB + Claude when installed.

* cascade_tool = ROADMAP (plant bottleneck / blast-radius) — designed & spec'd, not in the working prototype.

One loop

A single think→act→observe agent holds the full context and the entire tool suite — no sub-agents.

Dual mode

Identical tools run via Claude (LLM) or a deterministic Python pipeline — the reproducible offline baseline.

Smart tools

Every tool returns a rich structured object: RUL+SHAP, cited SOPs, stock+lead time, the exact SQL it ran.

One brain beats many agents

MULTI-AGENT (supervisor + sub-agents)

- Planning split across a supervisor and specialised agents
- "Split-brain": agents hold mismatched slices of context
- More moving parts, more orchestration overhead
- Harder to trace one coherent decision

CONSOLIDATED BRAIN (this project)

- One loop holds the full context + the entire tool suite
- No knowledge mismatch — nothing to reconcile
- Simpler, cheaper, fully observable trace
- Same five-stage capability, one accountable answer

Following the Omni / Blobby lesson: consolidate planning, don't fragment it.

Think → act → observe



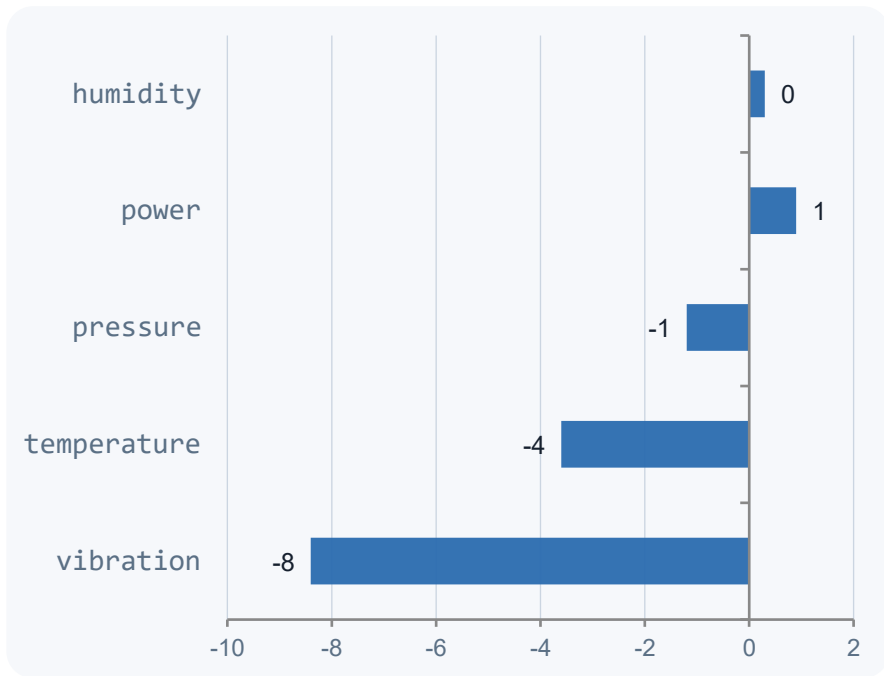
STEP 4.5 — Plant impact [ROADMAP — not in the prototype]: a designed cascade_tool would compute downstream blast radius so an asset's priority reflects what it idles, not just its own health.

Prognostics you can interrogate

Remaining Useful Life + 30-day failure probability, from a Random Forest trained on per-asset-type *deviations* (z-scores vs a healthy baseline). One model serves furnaces, pumps, valves...

SHAP attribution explains every prediction — negative bars push RUL down (accelerate failure). Falls back to model-agnostic ablation when the SHAP library is absent.

SHAP drivers — Mill Drive Gearbox 5 (example)



Constraint-aware prioritization

$$\text{Priority} = 100 \times (0.40 \cdot \text{RUL} + 0.30 \cdot \text{criticality} + 0.15 \cdot \text{delay-history} + 0.15 \cdot \text{spares})$$

deterministic, explainable, 0–100 → CRITICAL / HIGH / MEDIUM / LOW

The constraint flag

When an out-of-stock part's lead time exceeds the predicted RUL, the repair physically cannot finish before failure. The agent flips its own recommendation.

GEARBOX-05 — worked example

Predicted RUL: ~38 days

Pinion lead time: 45 days

→ **part arrives AFTER failure**

Recommendation flips: **"replace now"** → **monitored-degradation plan + auto-alert**

Production stack, zero-dependency fallback

Layer	Production path	Zero-dependency fallback
Prognostics	scikit-learn RandomForest	NumpyForest (bagged CART, numpy)
Explainability	shap.TreeExplainer	ablation attribution
Retrieval	ChromaDB vector store	TF-IDF cosine index (numpy)
Reasoning	Claude tool-use loop	deterministic Python pipeline

The deterministic path is also the reproducible baseline the eval judges score against — **the same fixed inputs always land in the same risk band.**

WHY IT STANDS OUT

Six things most submissions miss



Real prognostic ML

RUL + failure prob + SHAP — not just LLM-over-docs



Zero-hallucination

tool outputs are the only source of truth; SQL shown



Runs offline

deterministic mode + numpy fallback ladder



Multi-turn dialogue

context-aware follow-ups keep the asset in focus



Feedback loop

engineer corrections re-index RAG + adjust priority



Trace-based evals

judges guard every change against regressions

SEE IT WORK

Demo scenarios

“what's wrong with the mill gearbox?”

Constraint-aware triage

RUL < lead time → flips to monitored degradation + auto-alert

“that valve that keeps leaking on the caster”

Fuzzy resolution

plant jargon → HYD-VALVE-07 before any tool call

“check the EAF”

Abbreviation

resolves to FURNACE-01

“Pre-Shift Report tab”

Proactive autonomy

work orders drafted for every flagged asset before login

“Digital Logbook tab”

Loop closure

job closure blocked until all five compliance items are green

Roadmap — surpassing the requirements

Plant cascade graph

Designed

blast-radius / bottleneck propagation across the line

Domain-specific SLM

Planned

distilled maintenance model as an offline reasoning rung

Quantified outcomes

Planned

back-tested downtime-hours & cost avoided

Optimized scheduling

Planned

place repairs in the RUL window vs production calendar

FROM REACTIVE TO PROACTIVE

Predict. Prepare. Close the loop.

- Reduce unplanned downtime
- Faster, more consistent diagnosis
- Proactive, prioritized maintenance
- Audit-grade, traceable decisions

Maintenance Wizard · Consolidated-Brain agent for heavy steel plants